



University of Gondar

College of Informatics

Department of Computer Science

Introduction to Distributed System Group Assignment

Group 3

Assignment Title: A Comprehensive Analysis of the Berkeley Algorithm in
Distributed Systems

Group members

Name	ID number
1. Abebe Gashe	02459/15
2. Setey brhaney	00110/17
3. G/Mikael Aschale	03193/15
4. Aklile Tsige	02554/15
5. Melkamu Molla	02419/15

Submitted to: Mr. Aleka T

Date: 08/09/2018 E.C

Gondar, Ethiopia

Introduction to Clock Synchronization

When computers work together in a distributed system, they need to agree on the time to coordinate their tasks. However, every computer has its own physical clock inside it. Because of hardware differences, temperature changes, and normal wear and tear, these clocks don't tick at the exact same speed. Over time, the clocks will start to show different times. We call this problem "clock drift."

If we don't fix clock drift, the system can get really confused. For example, if two people edit a shared document, the system needs to know who saved it first. If the servers have different times, older data might overwrite newer data by mistake. To stop this from happening, we use clock synchronization algorithms.

One of the most famous methods to solve this is the Berkeley Algorithm. It was created in 1989 by Riccardo Gusella and Stefano Zatti. The Berkeley Algorithm is unique because it doesn't care about the "real world" time (like GPS or atomic clocks). Instead, it makes sure that all the computers inside the local network simply agree on the same average time.

How the Berkeley Algorithm Works

The Berkeley Algorithm uses a centralized "Master-Slave" setup. One computer is chosen to be the Master (the boss), and the rest are the Slaves.

Unlike other systems where slaves have to ask for the time, the Berkeley algorithm is active. The master takes control and regularly asks all the slaves what time they have.

Here is the step-by-step process of how it keeps everything synchronized:

- 1. Polling:** The master wakes up after a certain amount of time and sends a message to all the slave computers, asking for their current clock time.
- 2. Responding:** The slaves immediately check their clocks and send their times back to the master.
- 3. Fixing Network Delay:** Messages take time to travel through the network. The master calculates the Round Trip Time (RTT) by checking when it sent the request and when it got the reply. It divides this time by two to estimate the one-way travel delay, and adds this delay to the slave's time to make it accurate.

4. Removing Bad Clocks: Before finding the average, the master looks for broken clocks. If one slave's clock is way too fast or way too slow compared to the others, the master ignores it. This fault tolerance stops one broken computer from ruining the whole system.

5. Averaging: The master adds up all the valid slave times plus its own time, and finds the average.

6. Sending Adjustments: Instead of sending the new time as a strict timestamp, the master calculates the difference. For example, it tells Slave A to "add 4 seconds" and tells Slave B to "subtract 2 seconds." Sending offsets like this is safer because network delays won't mess up the adjustment.

Important Rules During Adjustment

There is a strict rule in computer systems: clocks should never go backward. If a slave receives a negative offset (meaning its clock is too fast), turning the clock backward could cause serious system errors. A file might look like it was saved before it was even created. To fix this, if the offset is negative, the slave simply slows down its clock's ticking speed until the real time catches up.

Handling Master Node Failures

Since the whole system relies on the master, what happens if the master computer crashes? This is a single point of failure.

To fix this, the system uses a "Leader Election" process. If the slave computers notice that the master hasn't asked for the time in a while, they realize the master is down. They will then use an election algorithm (like the Bully Algorithm) to vote among themselves. The computer with the highest IP address or longest uptime becomes the new master, and the synchronization process starts again.

The Mathematics of the Algorithm

To truly understand how the master calculates the adjustments, we can look at the math. Let's imagine a network with one Master (M) and three slaves (S_1, S_2, S_3).

1. The Master notes its own time: T_m
2. The Slaves report their times: T_1, T_2, T_3

3. The Master calculates the network delay for each message: d_1, d_2, d_3

First, the master adds the delay to the reported times to get the actual times:

- Adjusted S_1 time = $T'_1 (T_1 + d_1)$
- Adjusted S_2 time = $T'_2 (T_2 + d_2)$
- Adjusted S_3 time = $T'_3 (T_3 + d_3)$

Next, it checks for faulty clocks. Let's say T'_3 is completely wrong (an outlier). The master drops T'_3 from the calculation.

Now, it calculates the new average time (T_{avg}):

$$T_{avg} = (T_m + T'_1 + T'_2)/3$$

Finally, it calculates the exact offset (O) to send back:

- Master adjusts its own clock by: $O_m = T_{avg} - T_m$
- Offset sent to S_1 : $O_1 = T_{avg} - T'_1$
- Offset sent to S_2 : $O_2 = T_{avg} - T'_2$

Advantages and Disadvantages

Advantages:

- **No Internet Needed:** It doesn't rely on outside sources like NTP servers or UTC time. It works perfectly on closed, secure networks.
- **Fault Tolerant:** Because it drops outlier clocks, a single broken machine won't corrupt the rest of the network.
- **Smart Corrections:** By sending offsets instead of exact times, it avoids problems caused by network lag.

Disadvantages:

- **Single Point of Failure:** If the master node goes offline, sync stops until a new master is elected, which takes up time and network resources.
- **No Real-World Time:** The computers only agree with each other. If the whole network is 10 minutes slower than the real world, the algorithm doesn't know or care.

- **Traffic overload:** If there are thousands of computers on the network, the master can get overwhelmed trying to poll all of them at once. It works best on smaller, localized networks.

Real-World Applications

Because of how it works, the Berkeley algorithm is used in specific situations where internal agreement is more important than real-world time:

- **Isolated Networks:** Military and highly secure government databases that are disconnected from the internet use this to keep their servers matched up.
- **Car Computers:** Modern cars have dozens of internal computers (ECUs) controlling brakes, engines, and sensors. They use internal synchronization to talk to each other correctly without needing a GPS signal.
- **IoT Devices:** In a smart factory, hundreds of small sensors might not have a battery or internet to check the global time. A central gateway acts as the master to keep them all synced.

Comparing Berkeley to Cristian's Algorithm

To see why the Berkeley algorithm is useful, we can compare it to Cristian's Algorithm, which is another famous method.

In Cristian's algorithm, the roles are flipped. The slave computer is active, and the master is passive. The slave asks the master, "What time is it?" and the master simply replies with the time. Cristian's algorithm also assumes the master has access to the real-world UTC time.

The biggest danger with Cristian's algorithm is that if the master's clock is broken, it will blindly give the wrong time to every computer that asks, destroying the entire system. The Berkeley Algorithm fixes this exact issue because the master actively checks everyone's time, calculates an average, and filters out mistakes before making any changes. This makes Berkeley much safer for systems where hardware might fail.

Conclusion

The Berkeley Algorithm is a very important concept in distributed systems. Instead of relying on

the internet or external clocks, it builds a reliable internal timeline. Using an active master node, calculating network delays, and ignoring faulty outlier clocks, it provides a stable environment for computers to work together. Even though modern global systems use NTP, the logic behind the Berkeley Algorithm is still heavily used today in isolated networks, IoT systems, and fault-tolerant designs where relative consistency is all that matters.

References

- Coulouris, G., Dollimore, J., Kindberg, T., & Blair, G. (2011). Distributed Systems: Concepts and Design (5th ed.). Addison-Wesley.
- Gusella, R., & Zatti, S. (1989). The accuracy of the clock synchronization achieved by TEMPO in Berkeley UNIX 4.3BSD. IEEE Transactions on Software Engineering, 15(7), 847-853.